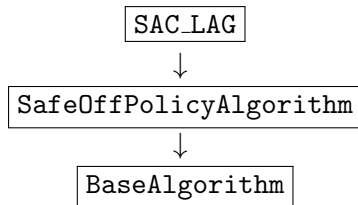


SAC-Lag: Implementation of Constrained SAC (Lagrangian method)

`sac_lag.py`

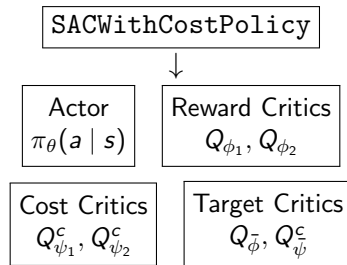
SAC-Lag Class Structure

Algorithm Hierarchy



`SAC_LAG` implements the safe RL training routine while inheriting the buffer and rollout infrastructure.

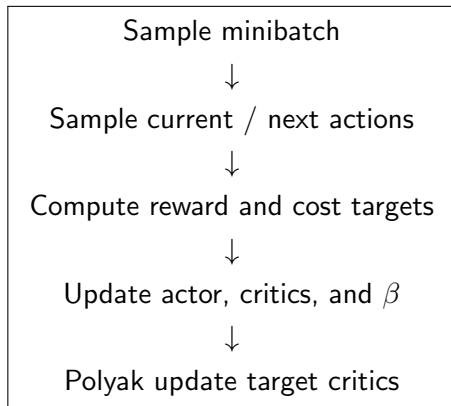
Policy Architecture



Repository: github.com/nrgrp/saferl-lib/tree/main

Training Step: Overview

Inside `train()`, each gradient step performs:



The important learned quantities are

$$\pi_{\theta}(a|s), \quad Q_{\phi_i}(s, a), \quad Q_{\psi_i}^c(s, a), \quad \beta$$

1. Sample a Minibatch

```
replay_data = self.replay_buffer.sample(  
    batch_size,  
    env=self._vec_normalize_env  
)
```

The sampled batch is

$$(s_t, a_t, r_t, c_t, s_{t+1}, d_t) \sim \mathcal{D}$$

where $\mathcal{D}$ is the replay buffer.

The important distinction:

$$a_t = \text{action stored in the replay buffer}$$

while later we sample a new action

$$a_t^\pi \sim \pi_\theta(\cdot | s_t).$$

2. Compute the Reward Target: Mathematics

The code implements the clipped double-Q SAC soft Bellman target:

$$y_t = r_t + \gamma(1 - d_t) \left[\min_{i=1,2} Q_{\bar{\phi}_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi_{\theta}(a_{t+1} | s_{t+1}) \right]$$

where the next action is sampled from the current policy:

$$a_{t+1} \sim \pi_{\theta}(\cdot | s_{t+1}).$$

Key properties:

- ▶ Calculated inside `th.no_grad()` (no backprop to target networks).
- ▶ $\min_i Q_{\bar{\phi}_i}$ prevents overestimation bias.
- ▶ Entropy bonus $-\alpha \log \pi_{\theta}$ encourages ongoing exploration.

2. Compute the Reward Target: Code

```
with th.no_grad():
    next_actions, next_log_prob = \
        self.actor.action_log_prob(
            replay_data.next_observations
        )

    next_q_values = th.cat(
        self.critic_target(
            replay_data.next_observations,
            next_actions
        ),
        dim=1
    )

    next_q_values, _ = th.min(
        next_q_values,
        dim=1,
        keepdim=True
    )

    next_q_values = \
        next_q_values \
        - ent_coef * next_log_prob.reshape(-1, 1)

    target_q_values = \
        replay_data.rewards \
        + (1 - replay_data.dones) \
        * self.gamma * next_q_values
```

3. Reward Critic Update

```
current_q_values = self.critic(  
    replay_data.observations,  
    replay_data.actions  
)  
  
critic_loss = 0.5 * sum(  
    [  
        F.mse_loss(current_q, target_q_values)  
        for current_q in current_q_values  
    ]  
)  
  
self.critic.optimizer.zero_grad()  
critic_loss.backward()  
self.critic.optimizer.step()
```

For each reward critic:

$$J_Q(\phi_i) = \frac{1}{2} \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[(Q_{\phi_i}(s, a) - y_t)^2 \right], \quad \phi_i \leftarrow \phi_i - \eta_Q \nabla_{\phi_i} J_Q.$$

4. Cost Critic Target and Update

```
next_qc_values = th.cat(  
    self.cost_critic_target(  
        replay_data.next_observations,  
        next_actions  
    ),  
    dim=1  
)  
next_qc_values, _ = th.max(next_qc_values, dim=1, keepdim=True)  
  
target_qc_values = \  
    replay_data.costs \  
    + (1 - replay_data.dones) * self.cost_gamma * next_qc_values
```

The cost Bellman target uses the **maximum** across critics (pessimism for safety):

$$y_t^c = c_t + \gamma_c(1 - d_t) \max_i Q_{\psi_i}^c(s_{t+1}, a_{t+1}), \quad J_{Q^c}(\psi_i) = \frac{1}{2} \mathbb{E}_{\mathcal{D}} \left[(Q_{\psi_i}^c(s_t, a_t) - y_t^c)^2 \right].$$

5. Lagrange Multiplier β

```
beta = F.softplus(self.soft_beta)

beta_loss = beta * sum(
    [
        th.mean(self.cost_constraint[0] - current_qc)
        for current_qc in current_qc_values
    ]
)

self.beta_optimizer.zero_grad()
beta_loss.backward(retain_graph=True)
self.beta_optimizer.step()
```

The implementation uses

$$\beta = \text{softplus}(\tilde{\beta}) \geq 0, \quad J_{\beta} = \beta \mathbb{E}_{\mathcal{D}} [d - Q^c(s, a)].$$

Thus β adapts according to the estimated cost relative to the constraint d .

Actor Loss: Exact Equation

The code

```
actor_loss = (  
    ent_coef * log_prob - min_qf_pi + beta * min_qc_pi  
).mean()
```

implements

$$J_{\pi}(\theta) = \mathbb{E}_{\substack{s \sim \mathcal{D} \\ a \sim \pi_{\theta}}} \left[\alpha \log \pi_{\theta}(a|s) - \min_i Q_{\phi_i}(s, a) + \beta \max_i Q_{\psi_i}^c(s, a) \right].$$

The three terms correspond directly to:

$$\underbrace{\alpha \log \pi_{\theta}}_{\text{entropy}} - \underbrace{\min_i Q_{\phi_i}}_{\text{reward}} + \underbrace{\beta \max_i Q_{\psi_i}^c}_{\text{cost penalty}}.$$

6. Actor Update

```
actions_pi, log_prob = self.actor.action_log_prob(
    replay_data.observations
)

q_values_pi = th.cat(
    self.critic.forward(replay_data.observations, actions_pi),
    dim=1
)

qc_values_pi = th.cat(
    self.cost_critic.forward(replay_data.observations, actions_pi),
    dim=1
)

min_qf_pi, _ = th.min(q_values_pi, dim=1, keepdim=True)
min_qc_pi, _ = th.max(qc_values_pi, dim=1, keepdim=True)

actor_loss = (
    ent_coef * log_prob
    - min_qf_pi
    + beta * min_qc_pi
).mean()
```

Actor Gradient Update

```
self.actor.optimizer.zero_grad()  
actor_loss.backward(retain_graph=True)  
self.actor.optimizer.step()
```

This is gradient descent on the actor objective:

$$\theta \leftarrow \theta - \eta_{\pi} \nabla_{\theta} J_{\pi}(\theta).$$

Because actions are reparameterized as $a_t^{\pi} = f_{\theta}(s_t, \epsilon_t)$, gradients flow end-to-end:

$$\theta \rightarrow a_t^{\pi} \rightarrow Q_{\phi}(s_t, a_t^{\pi}), \quad Q_{\psi}^c(s_t, a_t^{\pi}).$$

7. Polyak Averaging of Target Critics

```
polyak_update(  
    self.critic.parameters(),  
    self.critic_target.parameters(),  
    self.tau  
)  
  
polyak_update(  
    self.cost_critic.parameters(),  
    self.cost_critic_target.parameters(),  
    self.tau  
)
```

For the target critics:

$$\bar{\phi} \leftarrow \tau\phi + (1 - \tau)\bar{\phi}, \quad \bar{\psi} \leftarrow \tau\psi + (1 - \tau)\bar{\psi}.$$

No backpropagation is performed on target networks.

Complete Code → Equation Map

Code	Equation
<code>replay_buffer.sample()</code>	$(s, a, r, c, s') \sim \mathcal{D}$
<code>th.min(next_q_values)</code>	$\min_i Q_{\bar{\phi}_i}(s', a')$
<code>next_q - alpha * log_prob</code>	$Q - \alpha \log \pi$
<code>target_q_values</code>	$r + \gamma(Q - \alpha \log \pi)$
<code>MSE(current_q, target_q)</code>	$\frac{1}{2}(Q - y)^2$
<code>th.max(next_qc_values)</code>	$\max_i Q_{\psi_i}^c(s', a')$
<code>beta * (constraint - Qc)</code>	$\beta(d - Q^c)$
<code>actor_loss</code>	$\alpha \log \pi - Q + \beta Q^c$
<code>polyak_update()</code>	$\bar{\phi} \leftarrow \tau \phi + (1 - \tau) \bar{\phi}$

The Entire `train()` Function in Equations (1/2)

1. Minibatch Sampling & Next Actions

$$(s, a, r, c, s', d) \sim \mathcal{D}, \quad a' \sim \pi_{\theta}(\cdot | s')$$

2. Bellman Targets (Reward & Cost)

$$y = r + \gamma(1 - d) \left[\min_i Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_{\theta}(a' | s') \right], \quad y^c = c + \gamma_c(1 - d) \max_i Q_{\bar{\psi}_i}^c(s', a')$$

3. Critic Losses

$$J_Q = \frac{1}{2} \sum_i (Q_{\phi_i}(s, a) - y)^2, \quad J_{Q^c} = \frac{1}{2} \sum_i (Q_{\psi_i}^c(s, a) - y^c)^2$$

The Entire `train()` Function in Equations (2/2)

4. Actor Loss

$$J_{\pi} = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}} \left[\alpha \log \pi_{\theta}(a|s) - \min_i Q_{\phi_i}(s, a) + \beta \max_i Q_{\psi_i}^c(s, a) \right]$$

5. Multiplier Loss

$$J_{\beta} = \beta \left(d - \frac{1}{|I|} \sum_i Q_{\psi_i}^c(s, a) \right)$$

6. Target Network Polyak Averaging

$$\bar{\phi} \leftarrow \tau \phi + (1 - \tau) \bar{\phi}, \quad \bar{\psi} \leftarrow \tau \psi + (1 - \tau) \bar{\psi}$$